DATA 255 Deep Learning and Computer Vision in Applied Data Intelligence

Feb 25, 2026

ISB 870

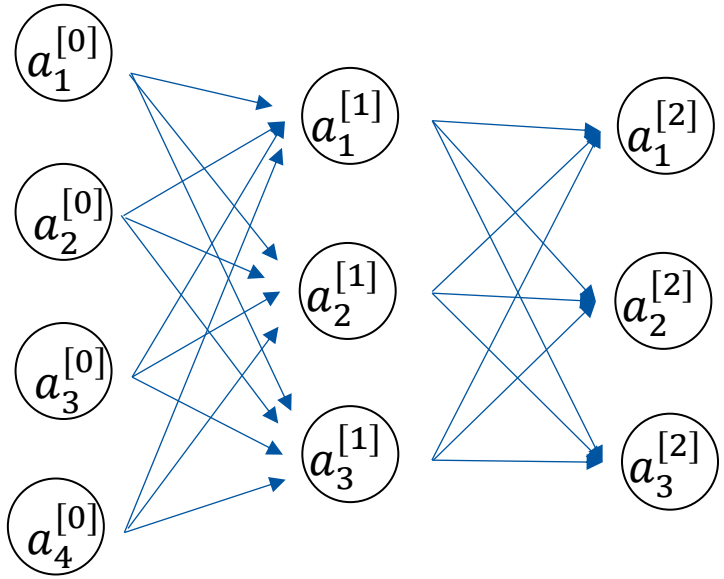
Taehee Jeong, Ph.D.

SJSU SAN JOSÉ STATE UNIVERSITY

Today's Topics

- Backward Pass for multiclass classification
- Learning rate
- Optimizer

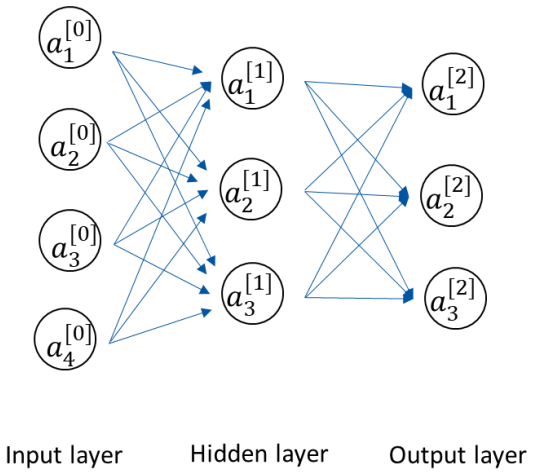
Neural Network: 3 output neurons



Input layer Hidden layer Output layer

Activation function of the last layer is *Softmax*.

Forward propagation



1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]} + W_{13}^{[1]} a_3^{[0]} + W_{14}^{[1]} a_4^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]} + W_{23}^{[1]} a_3^{[0]} + W_{24}^{[1]} a_4^{[0]}$$

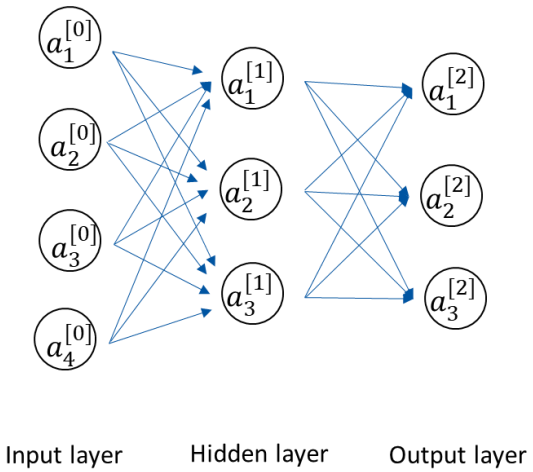
$$Z_3^{[1]} = W_{30}^{[1]} + W_{31}^{[1]} a_1^{[0]} + W_{32}^{[1]} a_2^{[0]} + W_{33}^{[1]} a_3^{[0]} + W_{34}^{[1]} a_4^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$a_3^{[1]} = \sigma(Z_3^{[1]})$$

Forward propagation



2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

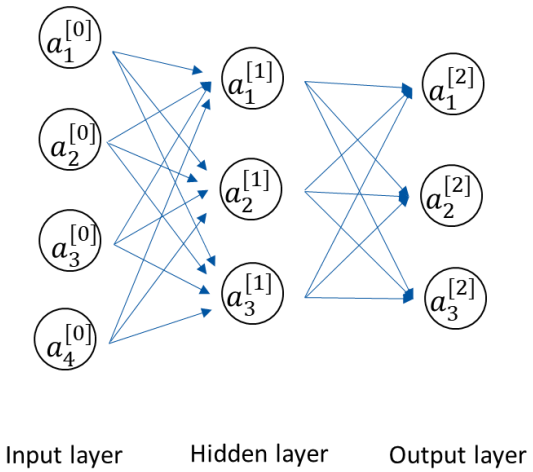
$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

$$a_1^{[2]} = S(Z_1^{[2]})$$

$$a_2^{[2]} = S(Z_2^{[2]})$$

$$a_3^{[2]} = S(Z_3^{[2]})$$

Forward propagation



$$a_1^{[0]}, a_2^{[0]}, a_3^{[0]}, a_4^{[0]}$$

$$\downarrow$$

$$Z_1^{[1]}, Z_2^{[1]}, Z_3^{[1]}$$

$$\downarrow$$

$$a_1^{[1]}, a_2^{[1]}, a_3^{[1]}$$

$$\downarrow$$

$$Z_1^{[2]}, Z_2^{[2]}, Z_3^{[2]}$$

$$\downarrow$$

$$a_1^{[2]}, a_2^{[2]}, a_3^{[2]}$$

Derivative of $a^{[2]}$

backpropagation for cross entropy loss

$$L(\hat{y}, y) = -[y_1 * \log \widehat{y_1} + y_2 * \log \widehat{y_2} + y_3 * \log \widehat{y_3}]$$

$$L(\hat{y}, y) = -[y_1 * \log\left(a_1^{[2]}\right) + y_2 * \log\left(a_2^{[2]}\right) + y_3 * \log(a_3^{[2]})]$$

Activation function of output layer is softmax

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\sum_{j=1}^{K} \exp(Z_j^{[2]})}$$

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

$$a_2^{[2]} = \frac{\exp(Z_2^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

$$a_3^{[2]} = \frac{\exp(Z_3^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

Derivative of $softmax$

$$S(Z_1) = \frac{\exp(Z_1)}{\exp(Z_1) + \exp(Z_2) + \exp(Z_3)}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i)) \qquad if\ i = j$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j) \qquad if\ i \neq j$$

Chain rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{d}{dx}\ h(x) = \left[\frac{d}{dx} f(x)\right] \cdot g(x) + f(x) \cdot \left[\frac{d}{dx} g(x)\right]$$

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} e^{-x} = -e^{-x}$$

$$a_1^{[2]} = S(Z_1^{[2]}) = \frac{\exp(Z_1^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i))$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j)$$

Derivative of $Z^{[2]}$

$$L(\hat{y}, y) = -[y_1 * \log\left(a_1^{[2]}\right) + y_2 * \log\left(a_2^{[2]}\right) + y_3 * \log(a_3^{[2]})]$$

$$L(\hat{y}, y) = -[y_1 * \log\left(S(Z_1^{[2]})\right) + y_2 * \log\left(S(Z_2^{[2]})\right) + y_3 * \log(S(Z_3^{[2]})]$$

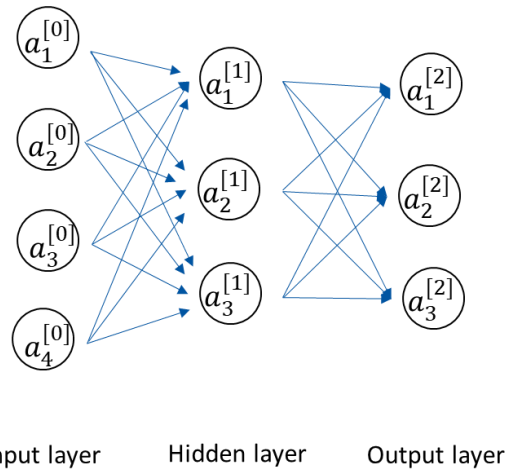
$$dZ_1^{[2]} = -\frac{y_1}{a_1^{[2]}} \cdot \frac{\partial S(Z_1^{[2]})}{\partial Z_1^{[2]}} - \frac{y_2}{a_2^{[2]}} \cdot \frac{\partial S(Z_2^{[2]})}{\partial Z_1^{[2]}} - \frac{y_3}{a_3^{[2]}} \cdot \frac{\partial S(Z_3^{[2]})}{\partial Z_1^{[2]}}$$

$$dZ_1^{[2]} = -y_1 \cdot (1 - a_1^{[2]}) + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + y_1 \cdot a_1^{[2]} + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + a_1^{[2]}$$

$$dZ_1^{[2]} = a_1^{[2]} - y_1$$



Forward propagation

$a_1^{[0]}, a_2^{[0]}, a_3^{[0]}, a_4^{[0]}$

$Z_1^{[1]}, Z_2^{[1]}, Z_3^{[1]}$

$a_1^{[1]}, a_2^{[1]}, a_3^{[1]}$

$Z_1^{[2]}, Z_2^{[2]}, Z_3^{[2]}$

$a_1^{[2]}, a_2^{[2]}, a_3^{[2]}$

Backward propagation

$a_1^{[2]}, a_2^{[2]}, a_3^{[2]}$

$dZ_1^{[2]}, dZ_2^{[2]}, dZ_3^{[2]}$

$dW_{10}^{[2]}, dW_{11}^{[2]}, dW_{12}^{[2]}, dW_{13}^{[2]}$

$dW_{20}^{[2]}, dW_{21}^{[2]}, dW_{22}^{[2]}, dW_{23}^{[2]}$

$dW_{30}^{[2]}, dW_{31}^{[2]}, dW_{32}^{[2]}, dW_{33}^{[2]}$

$da_1^{[1]}, da_2^{[1]}, da_3^{[1]}$

$dZ_1^{[1]}, dZ_2^{[1]}, dZ_3^{[1]}$

$dW_{10}^{[1]}, dW_{11}^{[1]}, dW_{12}^{[1]}, dW_{13}^{[1]}, dW_{14}^{[1]}$

$dW_{20}^{[1]}, dW_{21}^{[1]}, dW_{22}^{[1]}, dW_{23}^{[1]}, dW_{24}^{[1]}$

$dW_{30}^{[1]}, dW_{31}^{[1]}, dW_{32}^{[1]}, dW_{33}^{[1]}, dW_{34}^{[1]}$

Batch & Stochastic gradient descent

Batch Gradient descent

$$\frac{\partial J}{\partial W} = -\frac{1}{m}\sum_{i=1}^{m}[(\widehat{y^i} - y^i)x_j^i]$$

(Sum over data points)

Stochastic Gradient descent

$$\frac{\partial J}{\partial W} = -(\widehat{y^i} - y^i)x_j^i$$

(Each time, pick different data point)

Source: *Machine Learning, Emily Fox & Carlos Guestrin*

Mini-batch Gradient Descent

- Batch Gradient Descent

Computes the gradients based on full training set

Ex) Offline learning

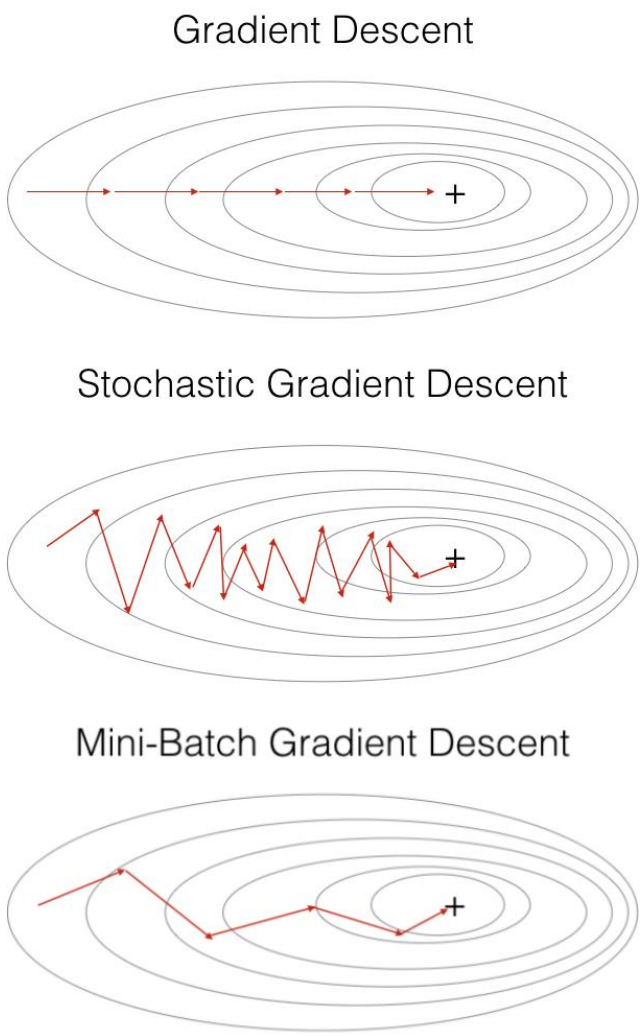
- Stochastic Gradient Descent

Computes just one instance

Ex) Online learning

- Mini-batch Gradient Descent

Computes the gradients on small random sets of instances

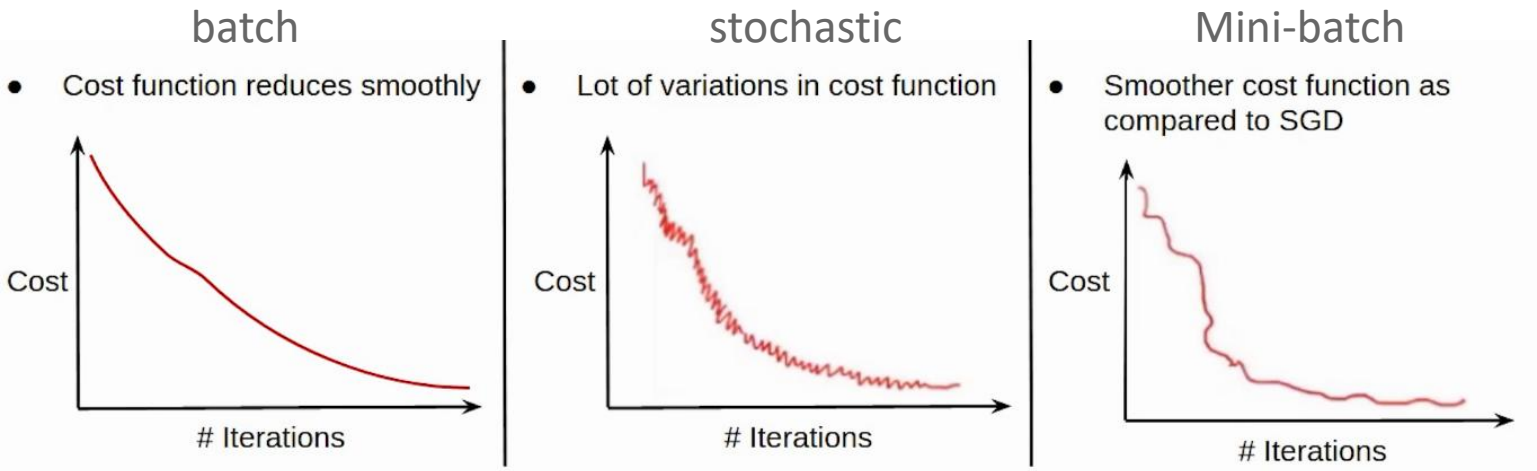


Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

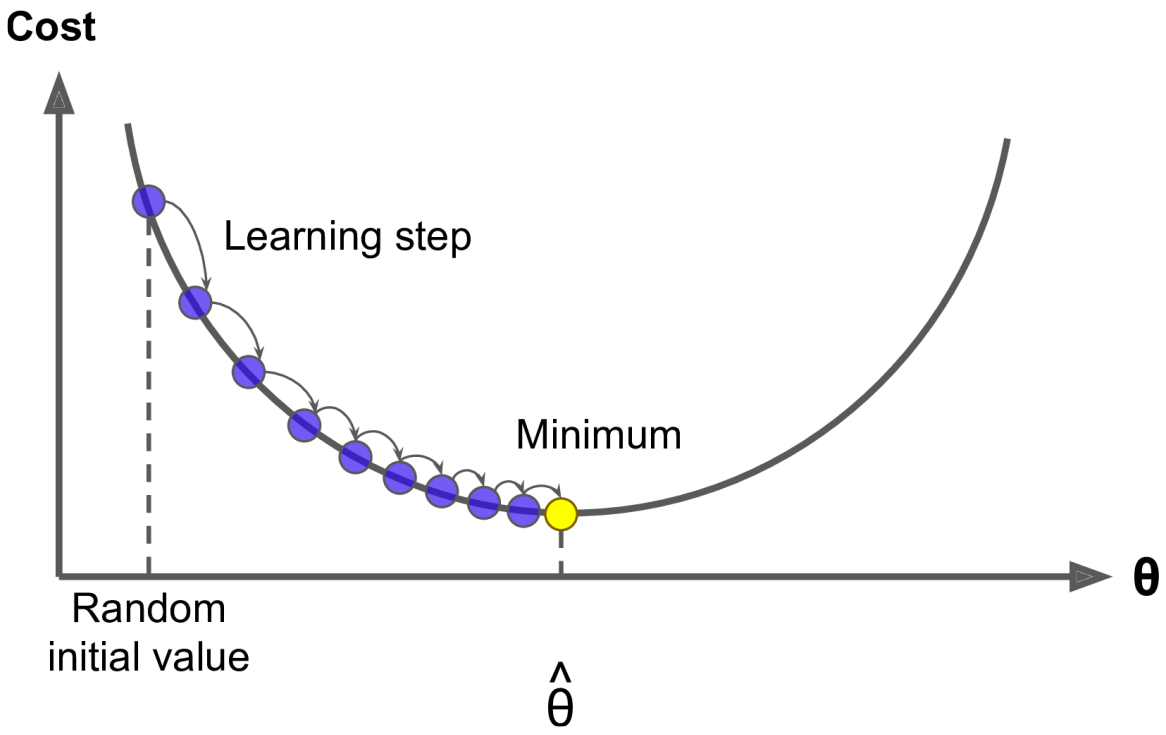
1 epoch : 1 pass through 1 batch, whole data set,
1 iteration : 1 pass through 1 mini-batch

Batch & Stochastic gradient descent

- " batch gradient descent" calculates the gradient using the entire training dataset for each parameter update
- "stochastic gradient descent" updates parameters based on the gradient calculated from just one randomly selected data point at a time, making it significantly faster for large datasets but potentially less stable due to the noisy updates



Gradient Descent



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

When Learning rate is too high

If learning rate is large, parameters decay too aggressively and the parameters are unable to reach the local minimum.

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

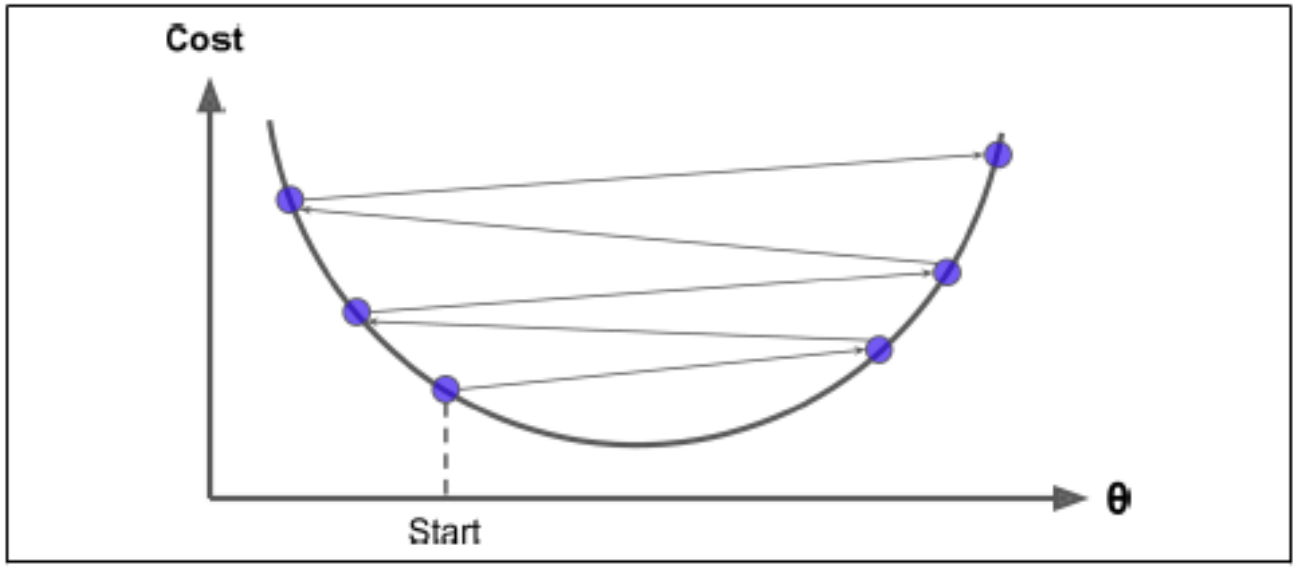


Figure 4-5. Learning rate too large

Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

Loss vs. Learning rate (or step size)

1 epoch : 1 pass through data set, iteration number

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Source: *Machine Learning, Emily Fox & Carlos Guestrin*

Learning rate decay

Slowing decreasing α

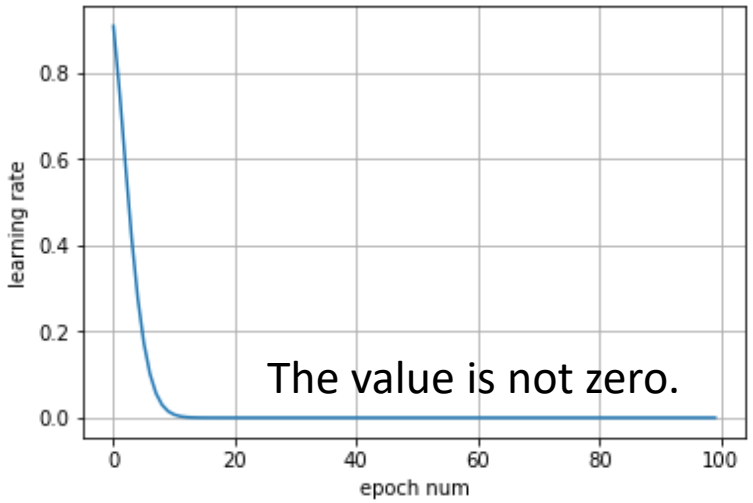
- 1/t decay
- Exponentially decay
- Step decay: discrete staircase
- Discrete staircase

1/t decay

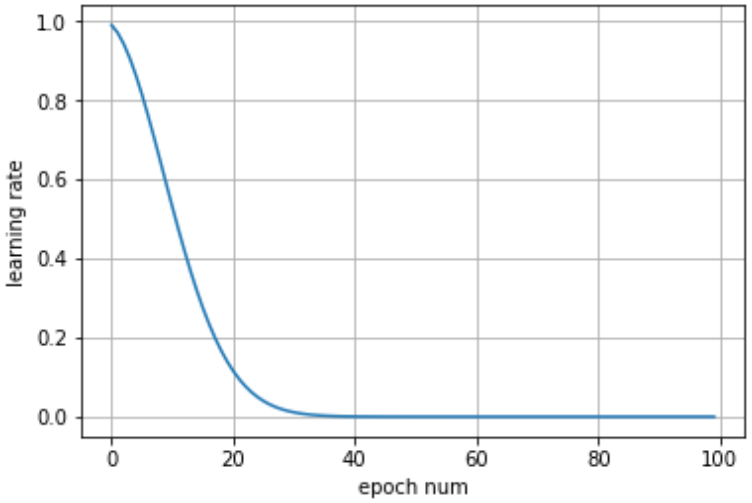
$$\alpha = \frac{\alpha_0}{1+decay_rate \times epoch_num} = \frac{\alpha_0}{1+k \cdot t}$$

$\alpha_0 = 1$

decay_rate(k) = 0.1



decay_rate(k) = 0.01

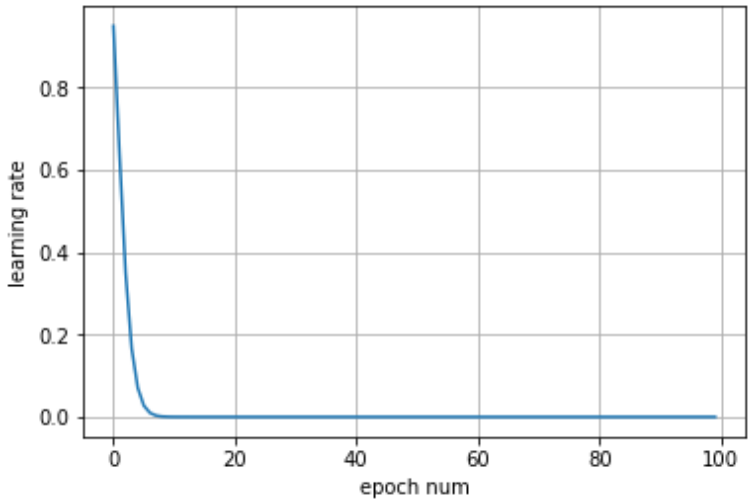


1/t decay

$$\alpha = \frac{k \times \alpha_0}{\sqrt{epoch_num}} = \frac{k \times \alpha_0}{\sqrt{t}}$$

$\alpha_0 = 1$

k= 0.95

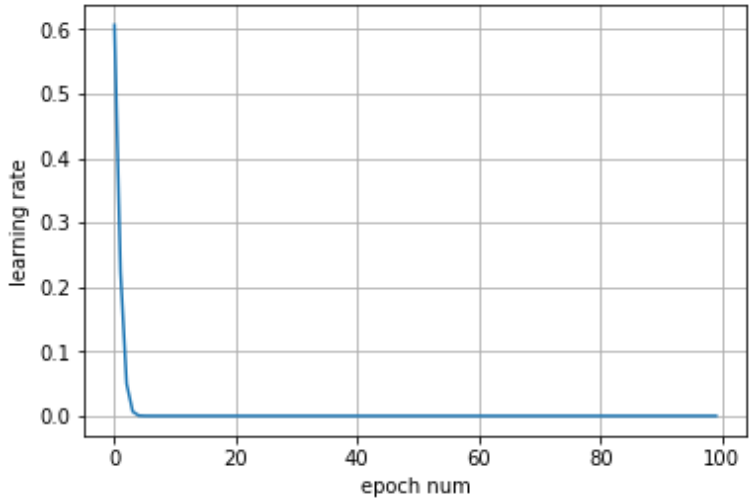


Exponential decay

$$\alpha = \alpha_0 \cdot \exp(-kt)$$

$\alpha_0 = 1$

k= 0.5

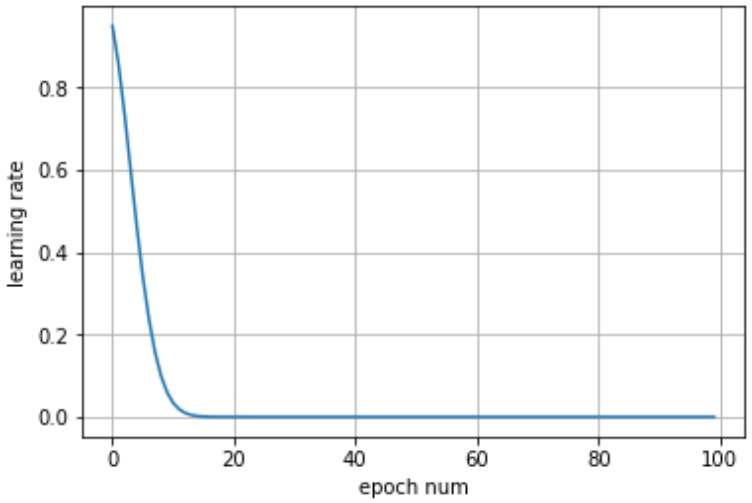


Exponential decay

$$\alpha = k^{epoch_num} \times \alpha_0 = k^t \times \alpha_0$$

$\alpha_0 = 1$

k= 0.95



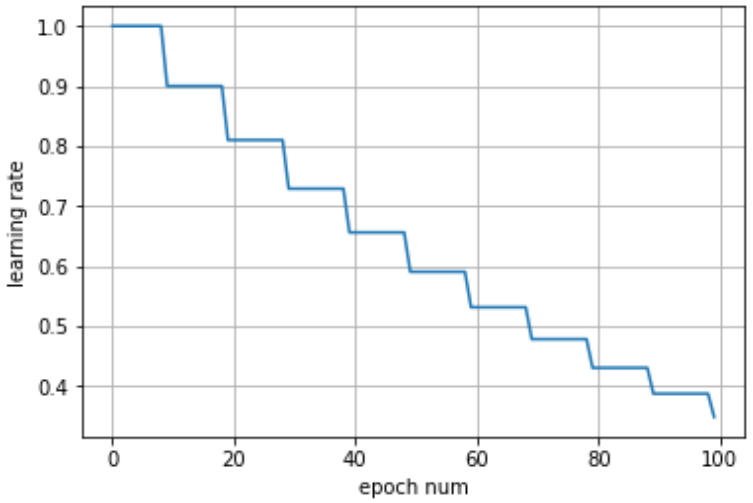
Discrete staircase

$\alpha = k \times \alpha$ with every few epochs

$\alpha_0 = 1$

k= 0.9

every 10 epochs



Learning rate decay

Slowing decreasing α

- 1/t decay: $\alpha = \frac{\alpha_0}{1 + kt}$ | $\alpha = \frac{k \times \alpha_0}{\sqrt{t}}$
- Exponentially decay

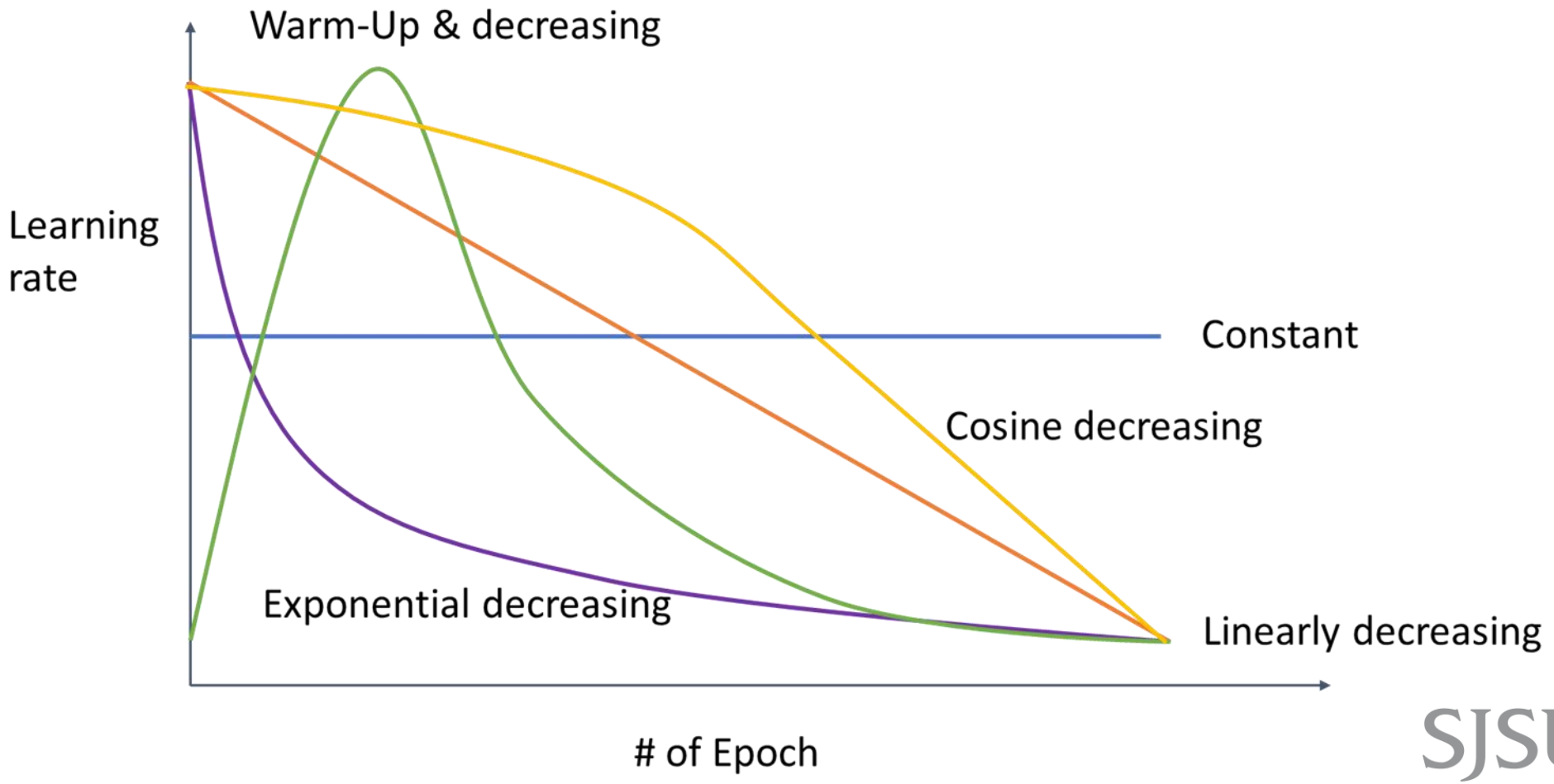
  $\alpha = \alpha_0 \cdot \exp(-kt)$ | $\alpha = k^t \times \alpha_0$
- Step decay: discrete staircase

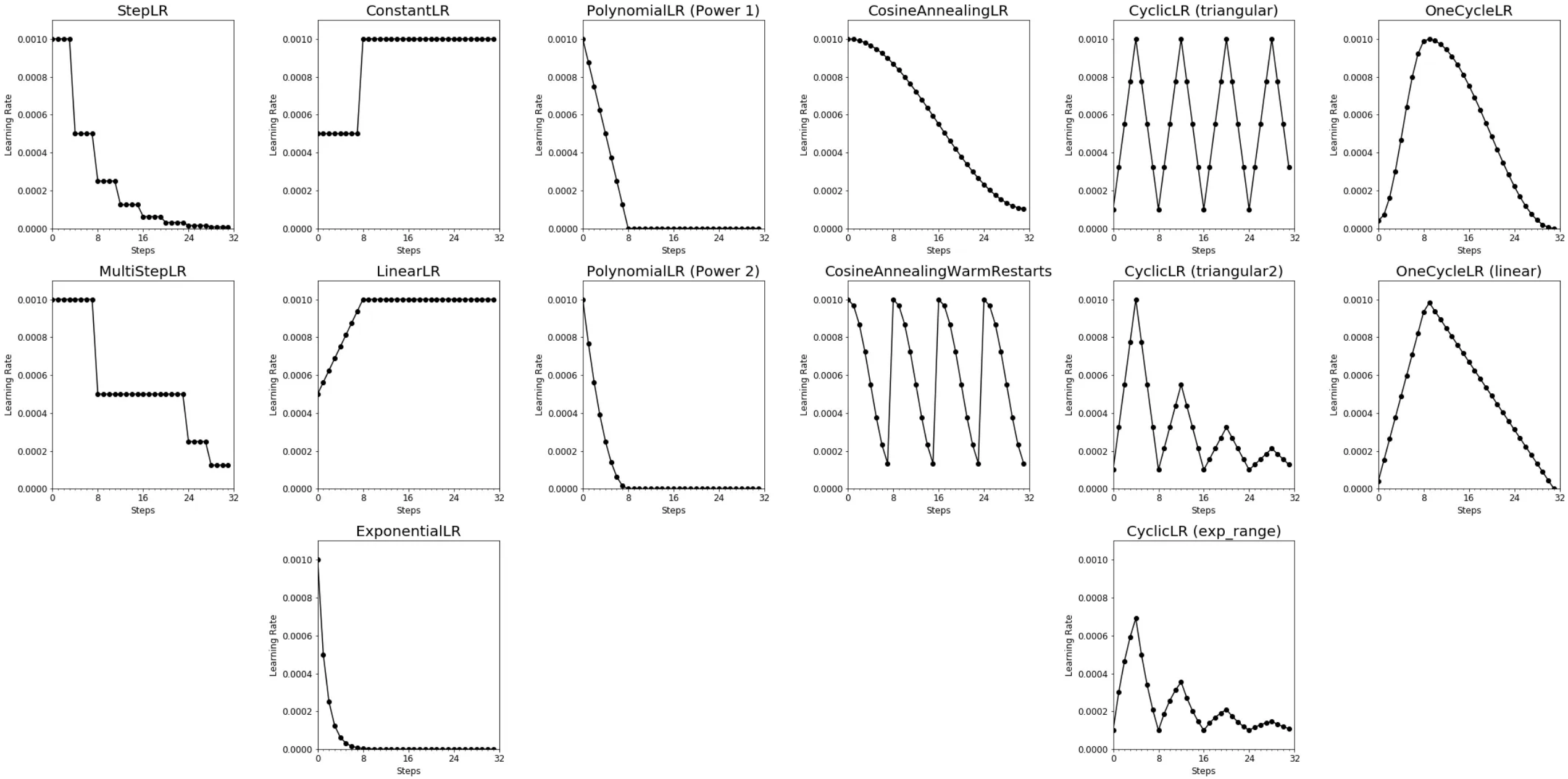
  $\alpha = k \times \alpha$ with every t

Choosing the right step size

- Using a fixed step size
- Gradually shrinking the step size over time
- Warm-up & shrinking

Learning rate over Epoch





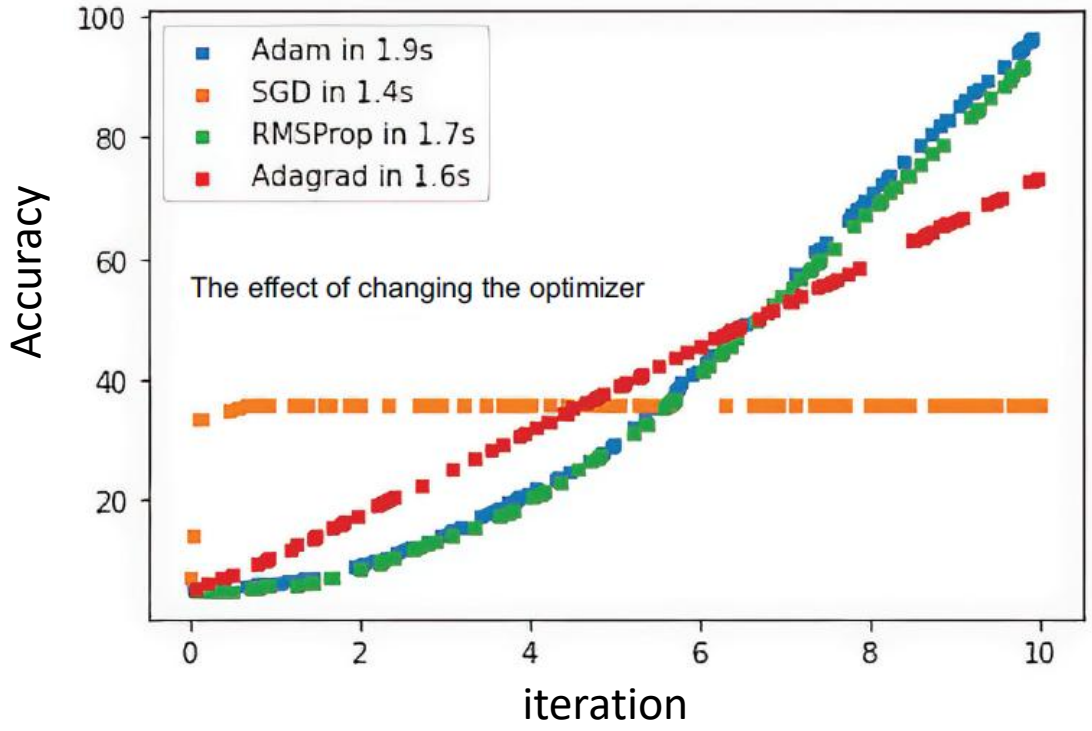
SJSU UNIVERSITY

Parameter update (Faster optimization)

- Momentum-based Gradient optimizer
- Nesterov momentum → not cover
- Exponentially Weighted Averages
- Adagrad
- Rmsprop
- Adam
- Second order method: based on Newton's method, L-BFGS → not cover

Source: An overview of gradient descent optimization algorithms

Comparison of optimizer functions

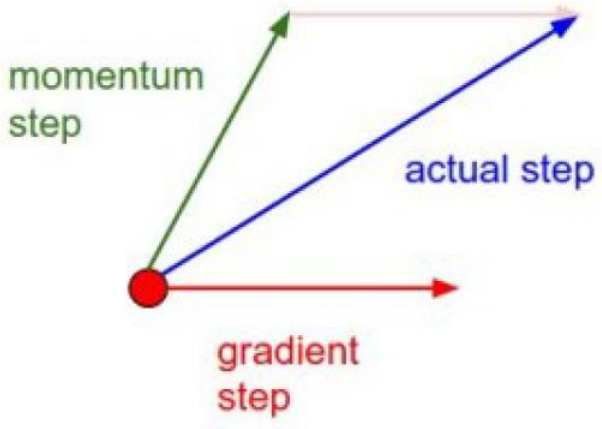


Source: Evolutionary Deep Learning, Micheal Lanham

Momentum update

Gradient descent update

x := x – learning rate*dx



Momentum update

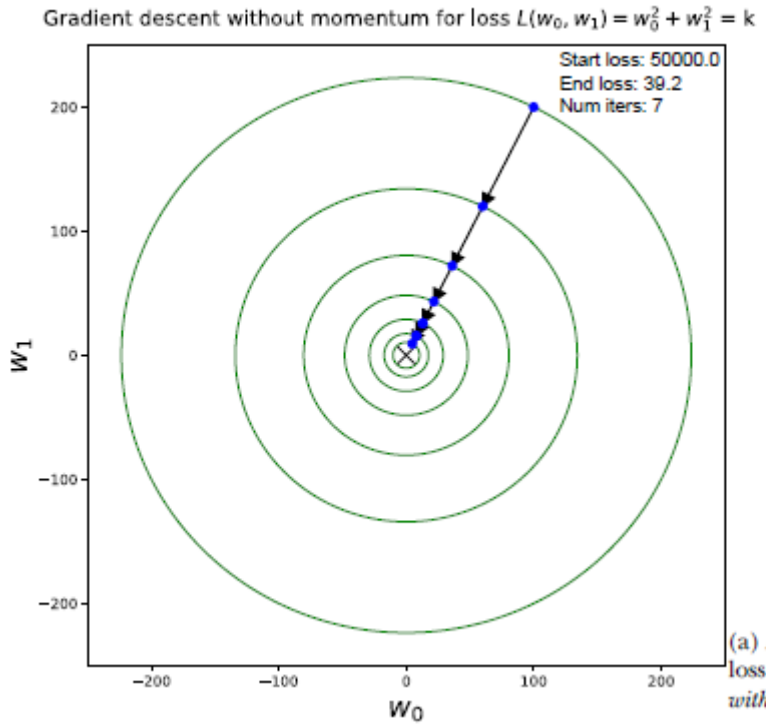
v = β*v – learning rate*dx

x := x + v

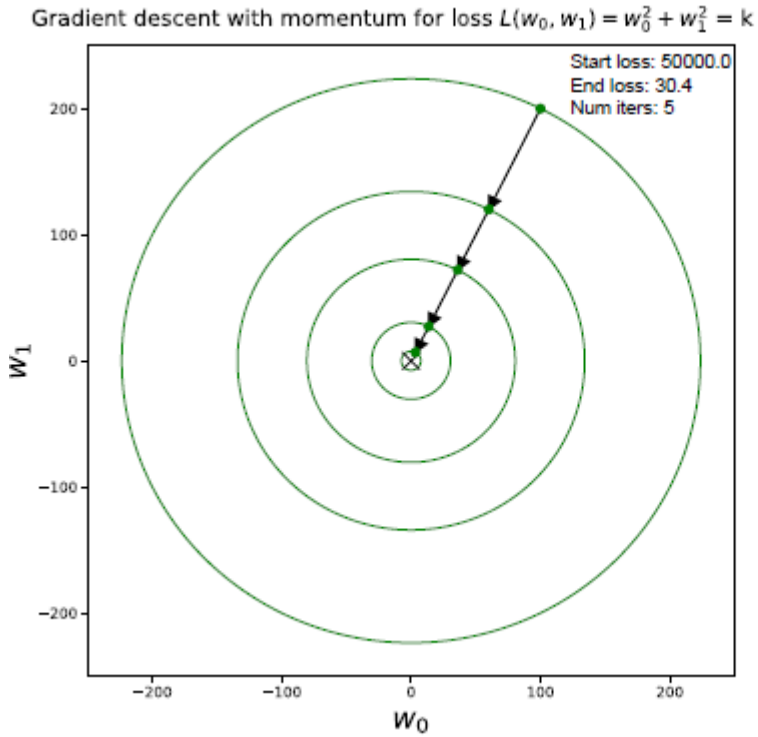
v : velocity

β : momentum

Gradient descent with/wo Momentum



(a) A trajectory through constant loss contours for gradient descent *without* momentum



(b) A trajectory through constant loss contours for gradient descent *with* momentum

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Adagrad

Adaptive learning rate method

$$v := v + dx^{**}2$$

$$x := x - learning_rate * dx / (sqrt(v) + eps)$$

v : sum of squared gradients

Normalized the parameter update step, element-wise.

Weights with high gradients will have effective learning rate reduces,

Weights with low gradients will have effective learning rate increases.

Gradient descent with momentum

$V_{dW} = 0, V_{db} = 0$

For every iteration t:
Compute V_{dW}, V_{db} using dW and db

$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$

$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$

$W := W - \alpha \cdot V_{dW}$

$b := b - \alpha \cdot V_{db}$

RMS prop

Root Mean square prop

$S_{dW} \neq 0, S_{db} \neq 0, \epsilon = 10^{-8}$

For every iteration t:
Compute S_{dW}, S_{db} using dW and db

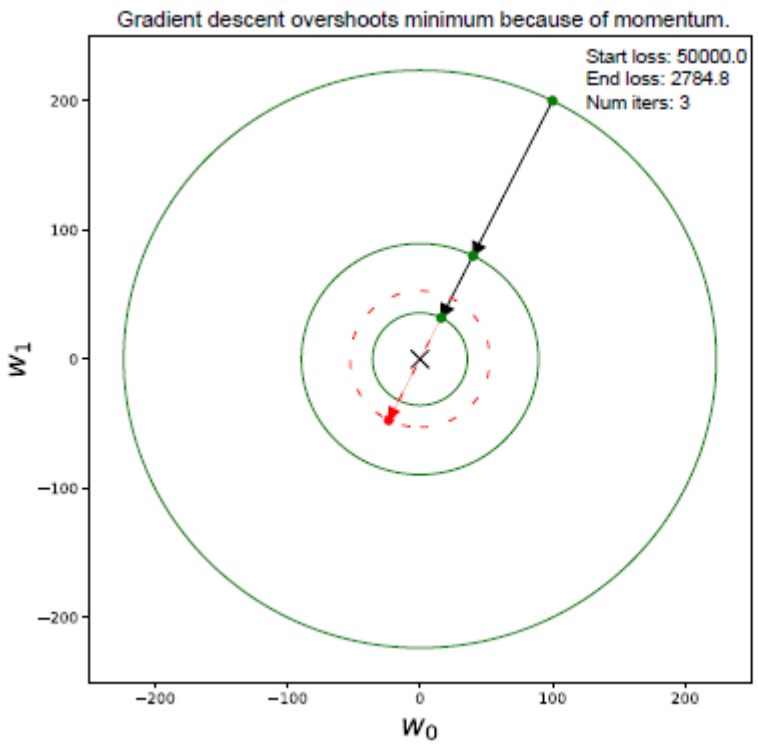
$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

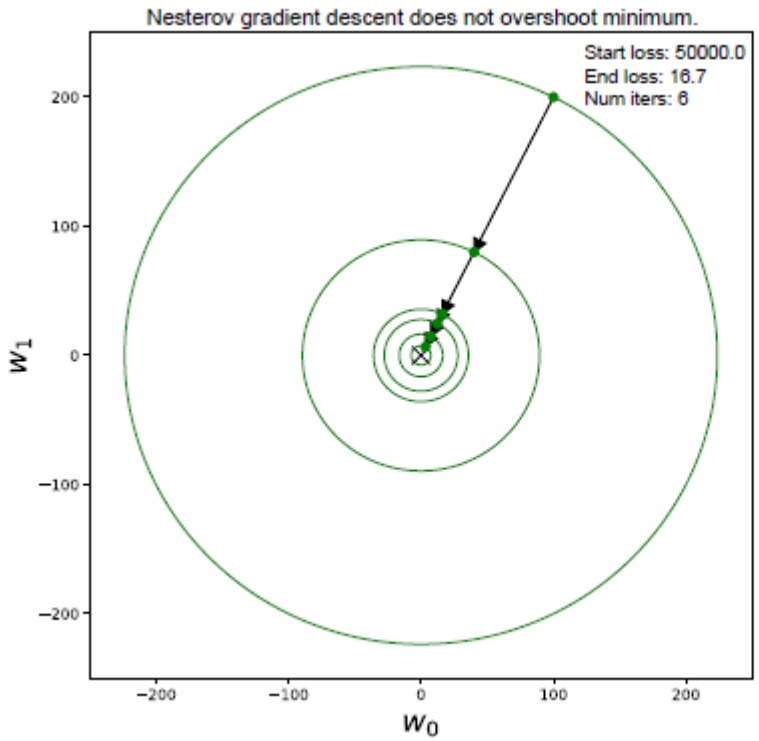
$$W := W - \alpha \cdot \frac{dW}{\sqrt{S_{dW} + \varepsilon}}$$

$$b := \mathrm{b} - \alpha \cdot \frac{db}{\sqrt{S_{db} + \varepsilon}}$$

Nesterov accelerated gradient



(a) Momentum gradient descent overshooting the minimum. Loss decreases for a while and then increases (dotted circle and arrow).



(b) Nesterov reduces the step size when we are about to overshoot the minimum.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Adam Optimization

Adaption moment estimation

$$V_{dW} = 0, V_{db} = 0, S_{dW} = 0, S_{db} = 0, \epsilon = 10^{-8}$$

For every iteration t:

Compute V_{dW}, V_{db} , S_{dW}, S_{db} using dW and db

$$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$$

$$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

$$W := W - \alpha \cdot V_{dW}$$

$$b := b - \alpha \cdot V_{db}$$

Adam Optimization

Bias correction

$$V_{dW}^{cor} = \frac{V_{dW}}{1-\beta_1^t}$$

$$V_{db}^{cor} = \frac{V_{db}}{1-\beta_1^t}$$

$$S_{dW}^{cor} = \frac{S_{dW}}{1-\beta_2^t}$$

$$S_{db}^{cor} = \frac{V_{db}}{1-\beta_2^t}$$

$$W := W - \alpha \cdot \frac{V_{dW}^{cor}}{\sqrt{S_{dW}^{cor} + \varepsilon}}$$

$$b := \mathrm{b} - \alpha \cdot \frac{V_{db}^{cor}}{\sqrt{S_{db}^{cor} + \varepsilon}}$$

Adam optimization

Hyper parameters choices

- α : need to be tune
- β_1 : 0.9 (momentum)
- β_2 : 0.999 (RMS prop)
- ε : 10^{-8}

Summary

SGD optimizer

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Momentum optimizer

$$m^t = \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} + m^t$$

$$W_j^t := W_j^{t-1} + \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Weighted Averages

$$m^t = \beta \cdot m^{t-1} + (1-\beta) \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} - \alpha \cdot m^t$$

$$W_j^t := W_j^{t-1} - \alpha \cdot [\beta \cdot m^{t-1} + (1-\beta) \frac{\partial J}{\partial W}]$$

Adagrad

$$v^t = \sum_{i=0}^{t-1} (\frac{\partial J}{\partial W^i})^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

RMSprop

$$v^t = \beta \cdot v^{t-1} + (1-\beta) \cdot (\frac{\partial J}{\partial W})^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

Adam

$$m^t := \beta_1 \cdot m^{t-1} + (1-\beta_1) \cdot \frac{\partial J}{\partial W}$$

$$v^t = \beta_2 \cdot v^{t-1} + (1-\beta_2) \cdot (\frac{\partial J}{\partial W})^2$$

$$W_j^t := W_j^{t-1} - \alpha \cdot \frac{\widehat{m^t}}{\sqrt{\widehat{v^t} + \varepsilon}} \cdot \frac{\partial J}{\partial W}$$

$$\widehat{m^t} = \frac{m^t}{1 - \{\beta_1\}^t}$$

$$\widehat{v^t} = \frac{v^t}{1 - \{\beta_2\}^t}$$